CRDC Submission Portal APIs

1. Introduction

This document walks you through the basics of submitting data to CRDC Submission Portal using APIs. Information provide in CRDC Data Submission Instructions will not be included in this document. It is recommended to get familiar with CRDC Data Submission Instructions and submission workflow using CRDC Data Submission Portal before interacting with CRDC Submission APIs.

CRDC Submission Portal provides a set of Graphql APIs (https://graphql.org/). There is only one API endpoint, https://hub.datacommons.cancer.gov/api/graphql. Different functions are provided as separate API queries (read) and mutations (write). Users may use API tools like Postman as client to interact with the APIs. Users may also choose to write code in any popular programming languages to interact with the APIs.

All CRDC Submission Portal API queries and mutations use HTTP POST method. The APIs will always return HTTP code 200 (success) even when API call fails. The API returns data and errors in JSON format. If an API call succeeds, the returned data will be under "**data**" key. If any error occurs, the error information will be returned under "**error**" key ("**data**" key will be null).

CRDC Submission Portal API is authentication and authorization controlled. API tool (or your code) needs to be set up to send "Bearer Token" in "**Authorization**" header.

A Graphql API's schema contains definitions of all queries, mutations and their parameters and return types. This document will only provide information that cannot be found in the Graphql schema.

2. Prerequisites

- A valid API token downloaded from Data Submission Portal.
- API testing tool, such as Postman or GraphiQL or custom code.
- Knowledge of using API tools and/or interacting with APIs in code.
- Basic knowledge of Graphql APIs (https://graphql.org/).

3. Conventions

A dot separated property name is used to describe the hierarchy in the data. For example, **submissions._id** means **_id** property under **submissions** property. See screenshot below.

```
"total": 24,
"submissions": [
    {
        "_id": "de9f113c-1a9d-4740-9495-2c595976b440",
        "name": "DELETE-1025_Delete_1201",
        "submitterName": "crdc.g.submtr",
```


4. Starting a new submission

Before creating a data submission, user needs to determine which study the data submission will be submitted to. To retrieve approved studies assigned to a user, call API query **getMyUser**.

API signature:

```
getMyUser : User
```

Sample Query	Sample graphql variables

query getMyUser {
getMyUser {
_id
studies {
_id
}
}
}	

Important return values:

- **studies._id**: should be used as **studyID** parameter of **createSubmission** API.

To create a new data submission, call API mutation **createSubmission**.

API signature:

```
createSubmission (
    studyID: String!,
    dataCommons: String!,
    name: String!,
    intention: String!,
    dataType: String!
): Submission
```

Sample Query	Sample graphql variables

mutation createSubmission($studyID: **String**!, $dataCommons: **String**!, $name: **String**!,$intention: **String**!, $dataType: **String**!) {
  createSubmission(
  studyID: $studyID
  dataCommons: $dataCommons
  name: $name
  intention: $intention
  dataType: $dataType
) {
  _id
  status
  createdAt
  }
}
	{
	  "studyID": "4ab75167-0121-4fea-b515-94c01d8380cc",
	  "dataCommons": "ICDC",
	  "name": "API full workflow",
	  "intention": "New/Update",
	  "dataType": "Metadata and Data Files"
	}

Important parameters:

- **studyID**: **_id** field of an approved study
- **dataCommons:** data common's name such as "CTDC", "ICDC", "PSDC" etc. For General Commons the name should be "CDS"
- **name**: a user selected name for the submission
- **intention**: should be one of ["New/Update", "Delete"]
- **dataType**: should be one of ["Metadata Only", "Metadata and Data Files"]

Important return values:

- **_id:** submission's ID, aka. submissionID

5. Continuing an existing submission

a. Retrieve a list of all submissions

To retrieve a list of all submissions a user has access to, call API query **listSubmissions.**

API signature:

```
listSubmissions(
  name: String,
  dbGaPID: String,
  dataCommons: String,
  submitterName: String,
  organization: String,
  status: [String],
  first: Int = -1,
  offset: Int = 0,
  orderBy: String = "updatedAt",
  sortDirection: String = "DESC"): ListSubmissions
```

Sample Query	Sample graphql variables

query listSubmissions($first: **Int**, $offset: **Int**, $orderBy: **String**,
$sortDirection: **String**, $status: [**String**]) {
listSubmissions(
first: $first
offset: $offset
orderBy: $orderBy
sortDirection: $sortDirection
status: $status
) {
total
submissions {
_id
name
submitterName
organization {
_id
name
}
dataCommons
studyAbbreviation
dbGaPID
modelVersion
status
conciergeName
createdAt
updatedAt
intention
dataFileSize {
formatted
}
}
}
}
	{
	"status": "All",
	"first": 2,
	"offset": 0,
	"sortDirection": "desc",
	"orderBy": "updatedAt"
	}

Important parameters:

- **Name:** results will be filtered by data submission's name
- **dbGaPID:** despite the name, this parameter filters submissions by either dbGaPID, study name or study abbreviation/acronym.
- **dataCommons:** results will be filtered by data submission's data commons
- **submitterName:** results will be filtered by data submission's creator's name
- **organization:** results will be filtered by data submission's organization name, ***this parameter is deprecated and will be removed in the future***
- **Status:** results will be filtered by this parameter if one or more of the following values is provided: ["New", "In Progress", "Submitted", "Released", "Completed", "Canceled", "Rejected", "Withdrawn", "Deleted", "All"]. If "All" is provided, no filter will be applied.

- **first:** number of records to be returned, if -1 is sent, API will return all available data
- **offset:** skip given number of records before returning data
- **orderBy:** property name used to sort returned data
- **sortDirection:** should be one of [“ASC”, “DESC”]

Important return values:

- **submissions._id:** submission’s ID, aka. submissionID

b. Retrieve information about a submission

To retrieve detailed information about a submission, call API query **getSubmission**.

API signature:

getSubmission(_id: ID!): Submission

Sample Query	Sample graphql variables

query getSubmission($id: **ID**!) {
getSubmission(_id: $id) {
_id
name
submitterID
submitterName
organization {
_id
name
}
dataCommons
modelVersion
studyID
studyAbbreviation
dbGaPID
bucketName
rootPath
status
metadataValidationStatus
fileValidationStatus
crossSubmissionStatus
validationStarted
validationEnded
validationScope
validationType
deletingData
fileErrors {
submissionID
type
validationType
	{
	"id": "eea2a531-4860-4e09-bf8e-151f73d4c379"
	}

batchID
displayID
submittedID
severity
uploadedDate
validatedDate
errors {
title
description
}
warnings {
title
description
}
}
history {
status
reviewComment
dateTime
userID
}
conciergeName
conciergeEmail
intention
dataType
otherSubmissions
createdAt
updatedAt
}
}	

Important parameters:

- **_id**: submission's ID, aka. submissionID

To retrieve statistics of a submission, call API query **submissionStats.**

API signature:

```
submissionStats(_id: ID!): SubmissionStats
```

Sample Query	Sample graphql variables

query getSubmission($id: **ID**!) {
submissionStats(_id: $id) {
stats {
nodeName
total
new
passed
warning
error
}
}
}
	{
	"id": "eea2a531-4860-4e09-bf8e-151f73d4c379"
	}

Important parameters:

- **_id**: submission's ID, aka. submissionID

To retrieve uploaded metadata, call API query **getSubmissionNodes.**

API signature:

```
getSubmissionNodes(
  submissionID: String!,
  nodeType: String!,
  status: String = "All",
  nodeID: String,
  first: Int = 10,
  offset: Int = 0,
  orderBy: String = "nodeID",
  sortDirection: String = "ASC"
): SubmissionNodes
```

Sample Query	Sample graphql variables

query getSubmissionNodes($_id: **String**!, $nodeType: **String**!,
$status: **String**, $submittedID: **String**, $first: **Int**, $offset: **Int**,
$orderBy: **String**, $sortDirection: **String**) {
getSubmissionNodes(
submissionID: $_id
nodeType: $nodeType
status: $status
nodeID: $submittedID
first: $first
offset: $offset
orderBy: $orderBy
sortDirection: $sortDirection
) {
total
IDPropName
properties
nodes {
nodeID
nodeType
status
props
}
}
}
	{
	"_id": "eea2a531-4860-4e09-bf8e-151f73d4c379",
	"first": 20,
	"offset": 0,
	"sortDirection": "desc",
	"orderBy": "studyID",
	"nodeType": "study",
	"status": "All",
	"submittedID": ""
	}

Important parameters:

- **submissionID**: submission's ID
- **nodeType**: type of the metadata node to be returned
- **status:** should be one of ["All", "New", "Error", "Passed", "Warning"]. If "All" is provided. no filter will be applied, otherwise, return will be filtered by metadata's status.
- **nodeID:** if provided, return will be filtered by provided node ID, any node ID partially match given value will be returned.
- **first:** number of records to be returned, if -1 is sent, API will return all available data
- **offset:** skip given number of records before returning data
- **orderBy:** property name used to sort returned data
- **sortDirection:** should be one of ["ASC", "DESC"]

Important return values:

- **IDPropName**: name of the metadata node's ID property
- **Properties**: names of all metadata node's properties
- **Nodes**.props: a JSON string contains all properties of the metadata node, needs to be parsed as JSON in the code.

To get a summary of the submission, call API query **getSubmissionSummary**

API signature:
getSubmissionSummary(submissionID: ID!): [nodeTypeSummary]

Sample Query	Sample graphql variables

query getSubmissionSummary($submissionID: ID!) {
getSubmissionSummary(submissionID: $submissionID) {
nodeType
new
updated
deleted
}
}
	{
	"submissionID": "e356d430-70e4-40eb-8bde-6265fff1b46c"
	}

Important parameters:

- submissionID: submission's ID

Important return values:

- nodeType: node type
- new: number of new records that will be added
- updated: number of existing records that will be updated
- deleted: number of existing records that will be deleted

To download dbGaP load sheet, call API query **downloadDBGaPLoadSheet**

API signature:
downloadDBGaPLoadSheet(submissionID:String!):String

Sample Query	Sample graphql variables

query downloadDBGaPLoadSheet($submissionID: String!) {
downloadDBGaPLoadSheet(submissionID: $submissionID)
}
	{
	"submissionID": "e356d430-70e4-40eb-8bde-6265fff1b46c"
	}

Important parameters:

- submissionID: submission's ID

Important return values:

- A pre-signed URL that can be used to download the dbGaP load sheets in zip format.

c. Edit a Submission

You can change the name of the submission by calling API **editSubmission.**

API signature:

```
editSubmission (
  _id: String!
  newName: String!
): Submission
```

Sample Query	Sample graphql variables

mutation EditSubmission($_id: String!, $newName: String!) {
editSubmission(_id: $_id, newName: $newName) {
_id
name
}
}
	{
	"_id": "*{{submissionID}}*",
	"newName": "My new Submission Name"
	}

Important parameters:

- **_id**: submission's ID
- **newName**: new submission name, maximum length is 25 characters.

d. Uploading Files and Manifests

It is recommended to upload data files via Uploader CLI Tool. It is also possible to upload data files by writing code.
To retrieve an CLI configuration file, call API query **retrieveCLIConfig**. Returned data contains correct content and format (including line breaks and indentation). It is recommended to call this API in code and save returned string into a YAML file without modifying the content in anyway. For example, "my-cli-configuration.yml". API tools have their own way of displaying data contains line breaks, it will be hard to preserve original return data in an API tool.

API signature:

```
retrieveCLIConfig(
  submissionID: String!,
  apiURL: String!,
  dataFolder: String,
  manifest: String
  archive_manifest: String
): String
```

Sample Query	Sample graphql variables

query retrieveCLIConfig($submissionID: String!, $apiURL: String!, $dataFolder: String, $manifest: String, $archive_manifest: String) {
retrieveCLIConfig(
submissionID: $submissionID
apiURL: $apiURL
dataFolder: $dataFolder
manifest: $manifest
archive_manifest: $archive_manifest
	{
	"submissionID": "0997c282-d3ac-47ab-a7f2-2dacd2ca7d7c",
	"dataFolder": "/Users/user1/Desktop/datafiles/",
	"manifest":"/Users/user1/Desktop/datafiles/manifest.tsv",
	"archive_manifest":"/Users/user1/Desktop/datafiles/archive_manifest.tsv",
	"apiURL": "https://hub-qa2.datacommons.cancer.gov/api/graphql"
	}

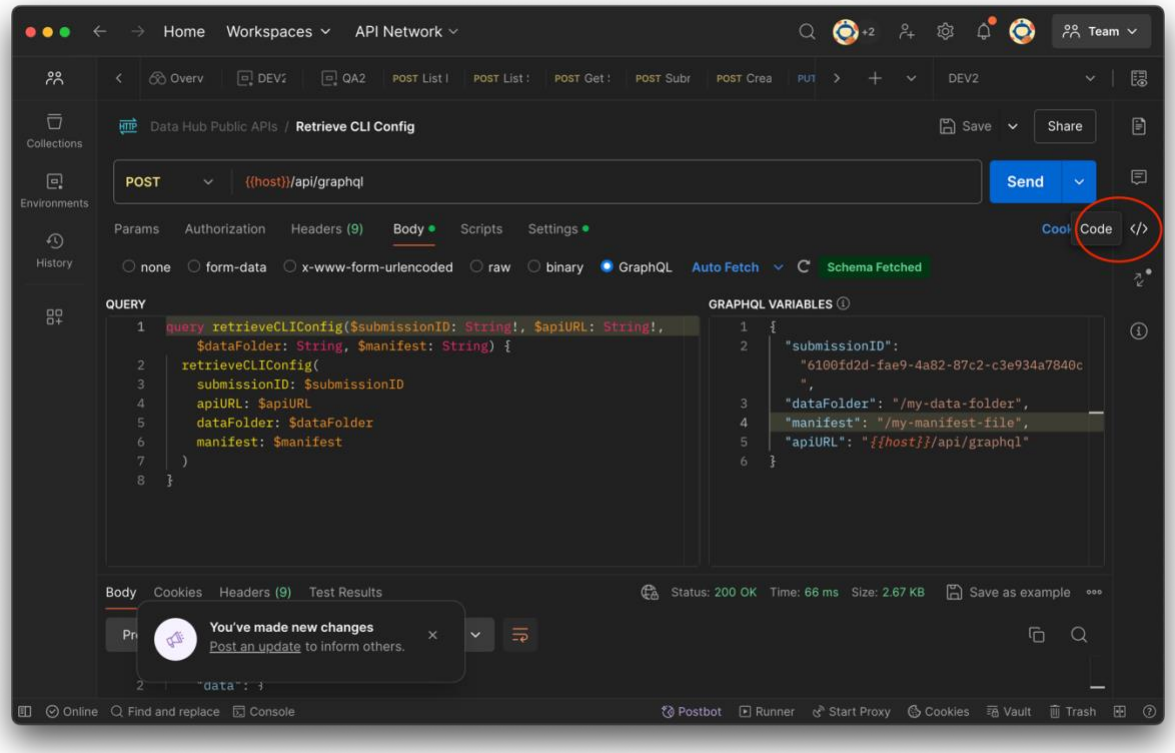
) }	

Important parameters:

- **submissionID**: submission's ID
- **apiURL**: must be **"https://hub.datacommons.cancer.gov/api/graphql"**
- **dataFolder**: local path of data files folder, for example *"/Users/me/my-data-files-folder"*
- **manifest**: local path of manifest file, for example *"/Users/me/my-metadata-folder/my-file-manifest.tsv"*
- **archive_manifest**: local path of archive manifest file, for example *"/Users/me/my-metadata-folder/archive-manifest.tsv"*

To save CLI configuration file retrieved as a file without writing code, follow steps below:

- Setup API request in Postman
- Click "Code" button (</>) on the right-side bar
- Copy and paste the command into a terminal (cmd on Windows), then add following code to end of the command
  " ***\| jq -r '.["data"]["retrieveCLIConfig"]' > my-cli-configuration.yml***"



If uploading data files in code is preferred, user may call API mutation **createTempCredentials** to retrieve a set of temporary credentials to use in the code.

API signature:

```
createTempCredentials (submissionID: ID!): TempCredentials
```

Sample Query	Sample graphql variables

mutation createTempCredentials ($submissionID: ID!) {
createTempCredentials(submissionID: $submissionID) {
accessKeyId
secretAccessKey
sessionToken
}
}
	{
	"submissionID": "6100fd2d-fae9-4a82-87c2-c3e934a7840c"
	}

Important parameters:

- **submissionID**: submission's ID

Metadata templates can be uploaded via Uploader CLI Tool. It is also possible to upload metadata templates via API. If using API is preferred, it is recommended to perform following steps in code rather than in API tool:

- Step 1: create a "batch" by calling API mutation **createBatch**.
- Step 2: upload metadata templates using pre-signed URLs retrieved in step 1.

- Step 3: update upload results by calling API mutation **updateBatch.** API will take the input as is and store the status of the files in the database. An asynchronous essential validation will be triggered by the API call, and the batch's status will be updated based on validation result. If validations passed, metadata will be loaded into submission database and status will be set to "**Uploaded**". Otherwise, batch status will be set to "**Failed**". Note, individual file status will be based on the input, and will not be updated by essential validation service.
- Step 4: retrieve essential validation results by calling API query **listBatches**

createBatch:

API signature:

```
createBatch (
  submissionID: ID!,
  type: String,
  files: [String!]!
): NewBatch
```

Sample Query	Sample graphql variables

mutation createBatch($submissionID: **ID**!, $type: **String**, $files: [**String**!]!) {
  createBatch(submissionID: $submissionID, **type**: $type, files: $files) {
  _id
  submissionID
  bucketName
  filePrefix
  type
  fileCount
  files {
  fileName
  signedURL
  }
  status
  createdAt
  updatedAt
  }
}
	{
	  "submissionID": "0997c282-d3ac-47ab-a7f2-2dacd2ca7d7c",
	  "type": "metadata",
	  "files": ["program.tsv", "study.tsv", "participant.tsv", "sample.tsv"]
	}

Important parameters:

- **submissionID**: submission's ID
- **type:** should be one of ["metadata", "data file"]
- **files**: list of file names to be uploaded.

Important return values:

- **_id**: batch's internal ID, aka batchID
- **files.signedURL**: S3 pre-signed URL that can be used to upload metadata templates

updateBatch:

API signature:

```
updateBatch (batchID: ID!, files: [UploadResult]): Batch
```

Sample Query	Sample graphql variables

`mutation updateBatch($batchID: ID!, $files: [UploadResult]) {`
`  updateBatch(batchID: $batchID, files: $files) {`
`  _id`
`  submissionID`
`  type`
`  fileCount`
`  files {`
`  filePrefix`
`  fileName`
`  size`
`  status`
`  errors`
`  createdAt`
`  updatedAt`
`  }`
`  status`
`  createdAt`
`  updatedAt`
`  }`
`}`
	`{`
	`  "batchID": "13af4bee-bed8-42ce-8252-eb5a756185b9",`
	`  "files": [`
	`  {`
	`  "fileName": "program.tsv",`
	`  "succeeded": true,`
	`  "errors": null`
	`  }, {`
	`  "fileName": "study.tsv",`
	`  "succeeded": true,`
	`  "errors": null`
	`  }, {`
	`  "fileName": "participant.tsv",`
	`  "succeeded": true,`
	`  "errors": null`
	`  }, {`
	`  "fileName": "sample.tsv",`
	`  "succeeded": true,`
	`  "errors": null`
	`  }`
	`  ]`
	`}`

Important parameters:

- **batchID:** batch's internal ID
- **files.skipped:** reserved for CLI use, should set to false

Important return values:

- **_id**: batch's internal ID, aka batchID
- **displayID**: batch's UI ID
- **status**: current status of the batch. It will change after essential validation is finished (explained in previous sections). Please call **listBatches** to get the latest status.

To retrieve information about all batches, call API query **listBatches**.

API signature:

```
listBatches(
 submissionID: ID!,
 first: Int = 10,
```

```
  offset: Int = 0,
  orderBy: String = "updatedAt",
  sortDirection: String = "DESC"
): ListBatches
```

Sample Query	Sample graphql variables

`query listBatches($submissionID: ID!, $first: Int, $offset: Int, $orderBy: String, $sortDirection: String) {`
`listBatches(`
`submissionID: $submissionID`
`first: $first`
`offset: $offset`
`orderBy: $orderBy`
`sortDirection: $sortDirection`
`) {`
`total`
`batches {`
`_id`
`displayID`
`createdAt`
`updatedAt`
`submissionID`
`type`
`fileCount`
`files {`
`nodeType`
`filePrefix`
`fileName`
`size`
`status`
`errors`
`createdAt`
`updatedAt`
`}`
`status`
`errors`
`}`
 	
`}`
 	
`}`
	`{`
	`"submissionID": "0997c282-d3ac-47ab-a7f2-2dacd2ca7d7c",`
	`"first": 20,`
	`"offset": 0,`
	`"sortDirection": "desc",`
	`"orderBy": "createdAt"`
	`}`

Important parameters:

- **submissionID**: submission's ID
- **first:** number of records to be returned, if -1 is sent, API will return all available data
- **offset:** skip given number of records before returning data
- **orderBy:** property name used to sort returned data
- **sortDirection:** should be one of ["ASC", "DESC"]

Important return values:

- **batches._id**: batch's internal ID, aka batchID
- **batches.displayID**: batch's UI ID
- **batches.files.nodeType:** node type contained in the metadata file. This value is only available when the metadata file can be successfully read by the essential validation service. Otherwise, it will be null.

e. Deleting data

To delete a metadata node, call API mutation **deleteDataRecords.**

API signature:

```
deleteDataRecords(
 submissionID: String!,
 nodeType: String!,
 nodeIDs: [String!],
        deleteAll: Boolean = false,
        exclusiveIDs: [String!] = [],
        deleteOrphanedDataFiles: Boolean = false
): DataValidation
```

Sample Query	Sample graphql variables

mutation deleteDataRecords($_id: String!, $nodeType: String!, $nodeIds: [String!], $deleteAll: Boolean, $exclusiveIDs: [String!]) {
deleteDataRecords(submissionID: $_id, nodeType: $nodeType, nodeIDs: $nodeIds, deleteAll: $deleteAll, exclusiveIDs: $exclusiveIDs, deleteOrphanedDataFiles: $deleteOrphanedDataFiles) {
success
message
}
}
	{
	"_id": "26000952-13c6-4cf5-9ac5-cf87b0c942f9",
	"nodeType": "sample",
	"nodeIds": [
	"allval_Samp_Sep18_01"
	],
	"deleteAll": false,
	"exclusiveIDs": [],
	"deleteOrphanedDataFiles: false
	}

Important parameters:

- **submissionID**: submission's ID
- **nodeType**: type of the metadata node to be deleted, or "**data file**" if deleting data files from S3 bucket is desired.
- **nodeIDs**: A list of node IDs. The API will delete metadata nodes that match the provided node IDs. This parameter is ignored if `deleteAll` is set to true.
- **deleteAll**: When set to true, all nodes of the given type will be deleted except those listed in the `exclusiveIDs` parameter.
- **exclusiveIDs**: Used with `deleteAll` to exclude specific nodes from being deleted.
- **deleteOrphanedDataFiles**: When set to true, system will delete data files if the associated file records are deleted. Please omit this parameter or set it to false if you want to preserve uploaded data files when deleting metadata.

Important return values:

- **success**: a Boolean value indicates if a deletion operation has been successfully initialized, the deletion will be performed asynchronously.

To delete a data file from submission bucket, call API mutation **deleteDataRecords.**

API signature:

```
deleteDataRecords(
 submissionID: String!,
 nodeType: String!,
 nodeIDs: [String!],
        deleteAll: Boolean = false,
        exclusiveIDs: [String!] = []
): DataValidation
```

Sample Query	Sample graphql variables

mutation deleteDataRecords($_id: String!, $nodeType: String!, $nodeIds: [String!], $deleteAll: Boolean, $exclusiveIDs: [String!]) {
deleteDataRecords(submissionID: $_id, nodeType: $nodeType, nodeIDs: $nodeIds, deleteAll: $deleteAll, exclusiveIDs: $exclusiveIDs, deleteOrphanedDataFiles: $deleteOrphanedDataFiles) {
success
message
}
}
	{
	"_id": "26000952-13c6-4cf5-9ac5-cf87b0c942f9",
	"nodeType": "data file",
	"nodeIds": [
	"41_batchtextfiles.txt"
	] ,
	"deleteAll": false,
	"exclusiveIDs": [],
	"deleteOrphanedDataFiles": false
	}

Important parameters:

- **submissionID**: submission's ID
- **nodeType**: must be "**data file**"
- **nodeIDs:** a list of file names to be deleted. This parameter is ignored if **`deleteAll`** is set to true.
- **deleteAll**: When set to true, all data files will be deleted except those listed in the **`exclusiveIDs`** parameter.
- **exclusiveIDs**: Used with **`deleteAll`** to exclude data files from being deleted.

Important return values:

- **success**: a Boolean value indicates if a deletion operation has been successfully initialized, the deletion will be performed asynchronously.

f. Running validations

To validate uploaded data, call API mutation **validateSubmission**.

API signature:

```
validateSubmission(
  _id: ID!,
  types: [String]
  scope: String
): DataValidation
```

Sample Query	Sample graphql variables

mutation validateSubmission($_id: **ID**!, $types: [**String**], $scope: **String**) {
validateSubmission(_id: $_id, types: $types, scope: $scope) {
success
}
}
	{
	"_id": "0997c282-d3ac-47ab-a7f2-2dacd2ca7d7c",
	"types": [
	"metadata", “data file”
	],
	"scope": "All"
	}

Important parameters:

- **_id**: submission’s ID, aka. submissionID
- **types:** any combination of following values: [“metadata”. “data file”]
- **scope:** should be one of [“New”, “All”]

Important return values:

- **success**: a Boolean value indicates if a validation has been successfully initialized, it has no relationship to the validation’s result.

To retrieve aggregated validation issues, call API query **aggregatedSubmissionQCResults.**

API signature:

```
aggregatedSubmissionQCResults(
  submissionID: ID!,
  severity: String = "all"
  first: Int = 20,
  offset: Int = 0,
  orderBy: String = "count"
  sortDirection: String = "DESC"): aggregatedQCResults
```

Sample Query	Sample graphql variables

query submissionQCResults($submissionID: **ID**!, $severity: **String**,
$first: **Int**, $offset: **Int**, $orderBy: **String**, $sortDirection: **String**) {
aggregatedSubmissionQCResults(
submissionID: $submissionID
severity: $severity
first: $first
offset: $offset
orderBy: $orderBy
sortDirection: $sortDirection
) {
total
results {
title
property
value
severity
count
code
}
}
}
	{
	"submissionID": "6100fd2d-fae9-4a82-87c2-c3e934a7840c",
	"first": -1,
	"offset": 0,
	"sortDirection": "desc",
	"orderBy": "property",
	"severity": "All"
	}

Important parameters:

- **submissionID**: submission's ID
- **severities**: should be one of ["all", "error", "warning"] , return will be filtered by given issue severity. "all" means both errors and warnings.
- **first:** number of records to be returned, if -1 is sent, API will return all available data
- **offset:** skip given number of records before returning data
- **orderBy:** should be one of ["count", "title", "property", "value", "code", "severity"]
- **sortDirection:** should be one of ["ASC", "DESC"]

Important return values:

- **title**: title of the issue
- **property**: property the issue related to
- **value**: the value that caused the issue
- **severity**: either "Error" or "Warning"
- **count**: number of occurrences of the issue
- **code**: issue code that can be used in **submissionQCResults** API as **issueCode** parameter

To retrieve detailed validation issues, call API query **submissionQCResults.**

API signature:

```
submissionQCResults(
  _id: ID!,
  nodeTypes: [String],
  batchIDs: [ID],
  severities: String,
  issueCode: String,

  first: Int = 10,
  offset: Int = 0
  orderBy: String = "uploadedDate",
  sortDirection: String = "DESC"
): QCResults
```

Sample Query	Sample graphql variables

query submissionQCResults($id: **ID**!, $nodeTypes: [**String**], $batchIDs: [**ID**], $severities: **String**, $first: **Int**, $offset: **Int**, $orderBy: **String**, $sortDirection: **String**) {
  submissionQCResults(
  _id: $id
  nodeTypes: $nodeTypes
  batchIDs: $batchIDs
  severities: $severities
  first: $first
  offset: $offset
  orderBy: $orderBy
  sortDirection: $sortDirection
) {
  total
  results {
  submissionID
  type
  validationType
  batchID
  displayID
  submittedID
  severity
  uploadedDate
  validatedDate
  errors {
  title
  description
  }
  warnings {
  title
  description
  }
  }
  }
}
	{
	  "id": "0997c282-d3ac-47ab-a7f2-2dacd2ca7d7c",
	  "first": -1,
	  "offset": 0,
	  "sortDirection": "desc",
	  "orderBy": "displayID",
	  "severities": "All"
	}

Important parameters:

- **_id**: submission's ID, aka. submissionID
- **nodeTypes:** a list of metadata node types or "data file", return will be filtered by given metadata node types or file validation results if "data file" is given.
- **batchIDs:** a list of batches' internal IDs, return will be filtered by given batch internal IDs
- **severities**: should be one of ["All", "Error", "Warning"] , return will be filtered by given issue severity. "All" means both errors and warnings.
- **issueCode:** results will be filtered by the issue code
- **first:** number of records to be returned, if -1 is sent, API will return all available data
- **offset:** skip given number of records before returning data
- **orderBy:** property name used to sort returned data
- **sortDirection:** should be one of ["ASC", "DESC"]

Important return values:

- **validationType**: either "metadata" or "data file"
- **submittedID:** a metadata node's ID, or file name of a data file

g. Submitting your Final Dataset

To submit a submission for review, call API mutation **submissionAction.**

API signature:

```
submissionAction (
  submissionID: ID!,
  action: String!
  comment: String
): Submission
```

Sample Query	Sample graphql variables

mutation submissionAction ($submissionID: **ID**!, $action: **String**!) {
submissionAction(submissionID: $submissionID, action: $action) {
_id
name
submitterID
submitterName
organization {
_id
name
}
dataCommons
modelVersion
studyID
studyAbbreviation
dbGaPID
	{
	"submissionID": "0997c282-d3ac-47ab-a7f2-2dacd2ca7d7c",
	"action": "Submit"
	}

bucketName
rootPath
status
metadataValidationStatus
fileValidationStatus
crossSubmissionStatus
validationStarted
validationEnded
validationScope
validationType
deletingData
fileErrors {
submissionID
type
validationType
batchID
displayID
submittedID
severity
uploadedDate
validatedDate
errors {
title
description
}
warnings {
title
description
}
}
history {
status
reviewComment
dateTime
userID
}
conciergeName
conciergeEmail
intention
dataType
otherSubmissions
createdAt
updatedAt
}
}	

Important parameters:

- **submissionID**: submission's ID
- **action:** should be "**Submit**" for this use case. Valid value includes ["Submit", "Withdraw", "Cancel"]